How to run application locally

To run the project locally, follow these steps:

Prerequisites

Node.js: Ensure you have Node.js installed. You can download it from nodejs.org.
Angular CLI: Install Angular CLI globally using npm:

npm install -g @angular/cli

.NET SDK: Ensure you have the .NET SDK installed. You can download it from dotnet.microsoft.com.
Steps

1. Clone the Repository
Clone the repository to your local machine:

git clone <repository-url>
cd <repository-directory>

2. Setup Angular Application
Navigate to the App directory and install the dependencies:

cd App
npm install

3. Run Angular Application
Start the Angular development server:

ng serve

Navigate to http://localhost:4200/ in your browser to see the application running.

4. Setup .NET API
Navigate to the NowApi directory and restore the .NET dependencies:

cd ../NowApi
dotnet restore

5. Run .NET API
Start the .NET API server:

dotnet run

The API will be available at https://localhost:7001 or http://localhost:5198.

Additional Information
Angular Configuration: The Angular project configuration can be found in angular.json.
.NET API Configuration: The .NET API configuration can be found in NowApi.csproj and launchSettings.json.

Architecture Design

The architecture of the project follows a clean architecture pattern, which promotes separation of concerns and maintainability. Here is a brief overview:

Layers
Presentation Layer (Angular)

Purpose: Handles the user interface and user interactions.
Technologies: Angular framework.
Components: Views, Components, Services.
Communication: Interacts with the API layer via HTTP requests.

API Layer (.NET)

Purpose: Exposes endpoints for the client applications and handles HTTP requests.
Technologies: ASP.NET Core.
Components: Controllers, DTOs (Data Transfer Objects).
Communication: Interacts with the Application layer to process requests.

Application Layer

Purpose: Contains the business logic and application-specific rules.
Components: Services, Use Cases, Commands, Queries.
Communication: Interacts with the Domain and Infrastructure layers.

Domain Layer

Purpose: Represents the core business logic and domain entities.
Components: Entities, Value Objects, Domain Services, Aggregates.
Communication: Independent of other layers, contains pure business logic.

Infrastructure Layer

Purpose: Handles data access, external services, and other infrastructure concerns.
Technologies: Entity Framework Core, external APIs.
Components: Repositories, Data Contexts, External Service Integrations.
Communication: Interacts with the Application layer to provide necessary data and services.

Flow of Data
User Interaction: User interacts with the Angular application.
HTTP Request: Angular application sends an HTTP request to the .NET API.
Controller Handling: The API layer's controller receives the request and forwards it to the Application layer.
Business Logic: The Application layer processes the request using business logic defined in the Domain layer.
Data Access: If needed, the Application layer interacts with the Infrastructure layer to fetch or persist data.
Response: The processed data is sent back through the layers to the Angular application, which updates the UI accordingly.
This architecture ensures a clear separation of concerns, making the application easier to maintain, test, and extend.